UNIVERSIDADE ESTADUAL DE GOIÁS – UEG
Bacharelado em Sistemas para Internet

Disciplina: Banco de Dados II
Professor: Natan

Relatório de Atividades Práticas:
Transações, Integridade, Segurança e Backup

Gustavo Borges Sousa

Goianésia, GO
13 de abril de 2026

Conteúdo

1 Introdução **3**

2 Parte 1: Controle de Concorrência e Transações **3**

3 Parte 2: Restrições de Integridade **4**

4 Parte 3: Segurança e Controle de Acesso **5**

5 Parte 4: Backup e Restauração **6**

6 Parte 5: Questões Teóricas (Sistema Acadêmico) **7**

7 Conclusão **8**

1 Introdução

Este relatório documenta a execução das atividades práticas propostas para a disciplina de Banco de Dados II. As tarefas foram realizadas em um ambiente isolado utilizando contêineres Docker (MySQL) e o sistema operacional Pop!_OS. O roteiro abrange a implementação de controle de concorrência via scripts Python, a definição de restrições de integridade no DBeaver, o gerenciamento de segurança e permissões de usuários, e, por fim, a realização de backups lógicos da base de dados.

2 Parte 1: Controle de Concorrência e Transações

O objetivo desta etapa foi demonstrar o isolamento de transações. Utilizou-se um script Python para simular a matrícula simultânea de dois alunos em uma disciplina com apenas 1 vaga disponível.

O script aplicou o comando `SELECT ... FOR UPDATE`, garantindo o bloqueio pessimista (lock) da linha na tabela `CURSOS`. Como demonstrado na Figura 1, a segunda transação foi colocada em uma fila de espera pelo SGBD. Após exceder o tempo limite estabelecido pelo MySQL (cerca de 50 segundos sem o *commit* da primeira transação), o sistema interrompeu a tentativa do segundo terminal para evitar *deadlocks*, retornando o erro `1205 (HY000): Lock wait timeout exceeded`.

```
Arquivo  Editar  Exibir  gustavo@pop-os: ~/Documentos/bd2-lista1/lista-4-concorrencia

gustavo@pop-os:~$ cd ~/Documentos/bd2-lista1/lista-4-concorrencia
source ../venv/bin/activate
python3 matricula_concorrente.py
Digite o ID do Aluno para este teste: 2

[Transação Iniciada] Aluno: 2
Consultando vagas e bloqueando a linha...
⚠ Erro: 1205 (HY000): Lock wait timeout exceeded; try restarting transaction
(venv) gustavo@pop-os:~/Documentos/bd2-lista1/lista-4-concorrencia$
```

```
Arquivo  Editar  Exibir  gustavo@pop-os: ~/Documentos/bd2-lista1/lista-4-concorrencia

gustavo@pop-os:~/Documentos/bd2-lista1/lista-4-concorrencia$ python3 preparar_banco.py
Traceback (most recent call last):
  File "/home/gustavo/Documentos/bd2-lista1/lista-4-concorrencia/preparar_banco.py", line 1, in <module>
    import mysql.connector
ModuleNotFoundError: No module named 'mysql'
gustavo@pop-os:~/Documentos/bd2-lista1/lista-4-concorrencia$ source ../venv/bin/activate
(venv) gustavo@pop-os:~/Documentos/bd2-lista1/lista-4-concorrencia$ python3 preparar_banco.py
Conectando ao MySQL...
Criando banco e tabelas do zero...
Inserindo você e a vaga teste...
✅ Tudo pronto! Pode testar a concorrência agora.
(venv) gustavo@pop-os:~/Documentos/bd2-lista1/lista-4-concorrencia$ python3 matricula_concorrente.py
Digite o ID do Aluno para este teste: 1

[Transação Iniciada] Aluno: 1
Consultando vagas e bloqueando a linha...
Curso: Banco de Dados II | Vagas disponíveis: 1

>>> BLOQUEIO ATIVO! <<<
Pressione ENTER para confirmar a matrícula e liberar o banco...
✅ Sucesso! Matrícula do aluno 1 realizada.
(venv) gustavo@pop-os:~/Documentos/bd2-lista1/lista-4-concorrencia$
```

Figura 1: Terminais evidenciando o bloqueio de transação e o erro de Timeout.

3 Parte 2: Restrições de Integridade

Nesta etapa, o foco foi a Integridade Referencial e de Domínio. As tabelas foram estruturadas utilizando o motor `InnoDB` para garantir o suporte total às chaves estrangeiras (*Foreign Keys*).

Foi criada a regra para impedir inserções na tabela de `MATRICULAS` caso o aluno ou o curso não existissem nas tabelas pai. Para comprovar a eficácia da restrição, realizou-se uma tentativa de *insert* de um "aluno fantasma"com `id_aluno = 999`. O SGBD bloqueou a ação com sucesso, retornando o erro `1452: Cannot add or update a child row`, conforme visualizado na Figura 2.

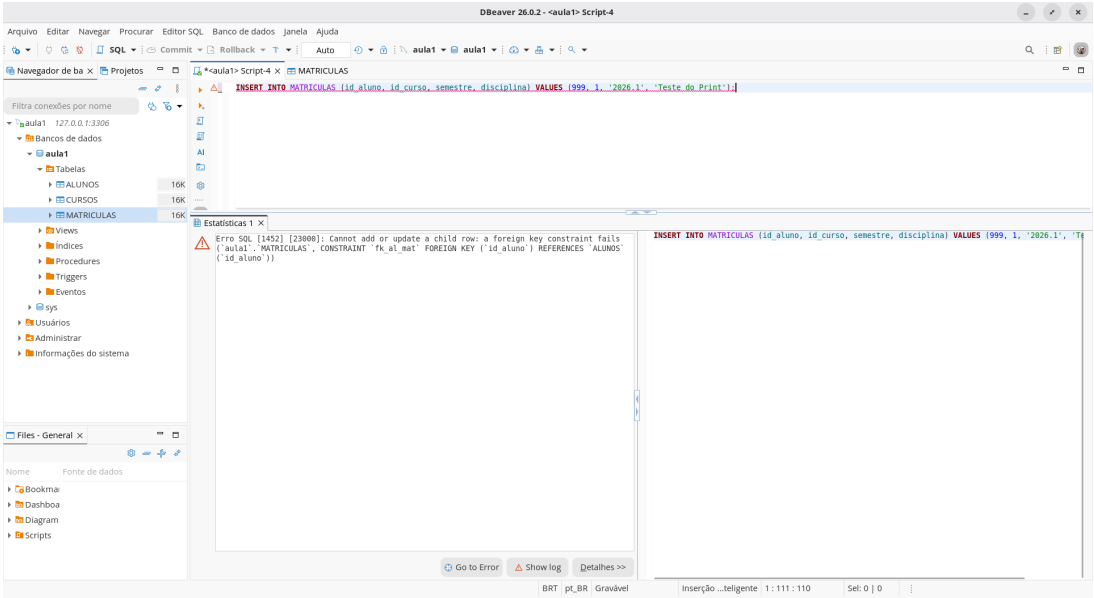

Figura 2: Erro emitido pelo DBeaver ao barrar a quebra de Integridade Referencial.

4 Parte 3: Segurança e Controle de Acesso

A segurança da base de dados foi configurada através da criação de papéis (usuários) específicos com o princípio do menor privilégio. Foram criados os usuários `aluno` e `professor`.

Utilizando os comandos `GRANT` e `REVOKE`, definiu-se que o usuário professor possuiria permissões restritas de leitura (`SELECT`) e alteração (`UPDATE`) na base de dados `aula1`. O comando `SHOW GRANTS FOR 'professor'@'%'` comprova a correta atribuição das permissões no dicionário de dados do MySQL.

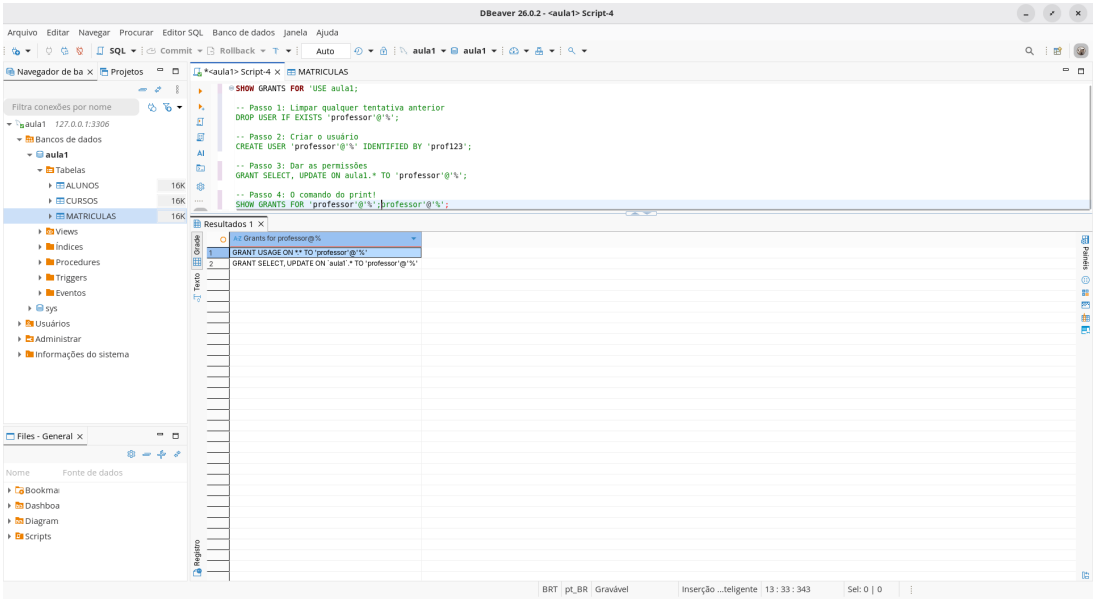

Figura 3: Demonstração das permissões ativas para o usuário professor.

5 Parte 4: Backup e Restauração

A última etapa validou as políticas de recuperação de desastres. Um backup lógico (*dump*) foi gerado diretamente através da interface de linha de comando interagindo com o contêiner Docker via `mysqldump`.

Figura 4: Arquivo de backup (.sql) gerado e inspecionado via terminal.

Em seguida, o banco de dados `aula1` foi intencionalmente excluído (`DROP DATABASE`) para simular uma perda de dados. A restauração foi conduzida com sucesso enviando o arquivo `.sql` gerado de volta ao SGBD. A Figura 5 ilustra o banco recriado com todos os seus registros e relacionamentos preservados.

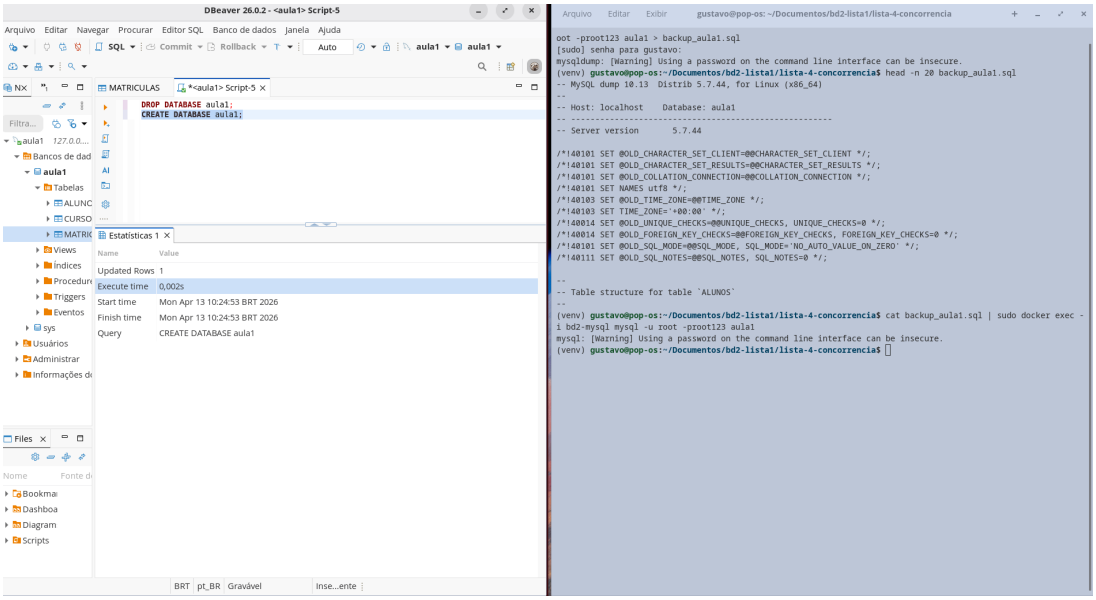

Figura 5: Banco de dados restaurado com sucesso após simulação de falha.

6 Parte 5: Questões Teóricas (Sistema Acadêmico)

Considerando o contexto do sistema acadêmico utilizado durante as aulas práticas desta disciplina, apresentam-se abaixo as definições teóricas solicitadas:

1. **Exemplo de problema de concorrência:** Um problema clássico é a "Atualização Perdida"(*Lost Update*) durante o período de matrículas. Se a disciplina possui apenas 1 vaga restante e dois alunos tentam se matricular no exato mesmo milissegundo, ambas as transações podem ler o valor de vagas como 1. Se o SGBD não aplicar mecanismos de bloqueio (como o *lock* utilizado na Parte 1), ambas as transações aprovarão a matrícula e atualizarão as vagas para 0, resultando em um aluno matriculado a mais do que o limite físico da turma (inconsistência).

2. **Regra de segurança:** A aplicação do Princípio do Menor Privilégio. No sistema, um usuário autenticado com o papel `aluno` deve ter permissão apenas de leitura (`GRANT SELECT`) para visualizar as disciplinas disponíveis e suas próprias notas, sendo expressamente proibido de executar comandos de `INSERT`, `UPDATE` ou `DELETE`. Já um usuário `professor` pode ter permissão de `UPDATE` na tabela de notas, mas sem permissão para excluir (`DELETE`) o registro de um aluno do banco de dados.

3. **Regra de integridade:** A Integridade Referencial, garantida através de Chaves Estrangeiras (*Foreign Keys*). Uma regra vital é que a tabela `MATRICULAS` não pode aceitar a inserção de um registro se o código de identificação (`id_aluno`) não existir previamente na tabela `ALUNOS`. Isso impede a criação de matrículas associadas a "alunos fantasmas". Adicionalmente, uma restrição de domínio (`CHECK (vagas >= 0)`) garante que o sistema jamais registre um número negativo de vagas disponíveis em um curso.

4. **Cenário de falha que exija recuperação:** Um erro humano catastrófico (como um administrador executando um comando `DROP TABLE ALUNOS` ou `DROP DATABASE` acidentalmente no ambiente de produção, como simulado na Parte 4) ou uma falha

de *hardware* que corrompa o disco físico do servidor. Nesses cenários, o banco de dados perde as informações ativas. A recuperação exige a utilização do último *backup* lógico válido para restaurar a estrutura das tabelas e repovoar os dados, minimizando o impacto acadêmico e a perda de histórico.

7 Conclusão

As atividades permitiram consolidar na prática os conceitos teóricos de Administração de Banco de Dados. A vivência com erros reais (como falhas de *Storage Engine* e controle do `foreign_key_checks`) e bloqueios transacionais reforçou a importância do papel do DBA na manutenção da consistência, segurança e disponibilidade dos dados corporativos.